Although many metrics can be defined for technical reviews, a relatively small subset can provide useful insight. The following review metrics can be collected for each review that is conducted:

- **Preparation effort,** E_p. The effort (in person-hours) required to review a work product prior to the actual review meeting
- **Assessment effort,** E_a. The effort (in person-hours) that is expended during the actual review
- **Rework effort,** E_r. The effort (in person-hours) that is dedicated to the correction of those errors uncovered during the review
- **Review effort,** E_{review}. Represents the sum of effort measures for reviews:

  $$E_{\text{review}} = E_p + E_a + E_r$$

- **Work product size (WPS).** A measure of the size of the work product that has been reviewed (e.g., the number of UML models, the number of document pages, or the number of lines of code)
- **Minor errors found,** $\text{Err}_{\text{minor}}$. The number of errors found that can be categorized as minor (requiring less than some prespecified effort to correct)
- **Major errors found,** $\text{Err}_{\text{major}}$. The number of errors found that can be categorized as major (requiring more than some prespecified effort to correct)
- **Total errors found,** Err_{tot}. Represents the sum of the errors found:

  $$\text{Err}_{\text{tot}} = \text{Err}_{\text{minor}} + \text{Err}_{\text{major}}$$

- **Error density.** Represents the errors found per unit of work product reviewed:

  $$\text{Error density} = \frac{\text{Err}_{\text{tot}}}{\text{WPS}}$$

How might these metrics be used? As an example, consider a requirements model that is reviewed to uncover errors, inconsistencies, and omissions. It would be possible to compute the error density in several different ways. Assume the requirements model contains 18 UML diagrams as part of 32 overall pages of descriptive materials. The review uncovers 18 minor errors and 4 major errors. Therefore, $\text{Err}_{\text{tot}} = 22$. Error density is 1.2 errors per UML diagram, or 0.68 errors per requirements model page.

If reviews are conducted for a number of different types of work products (e.g., requirements model, design model, code, test cases), the percentage of errors uncovered for each review can be computed against the total number of errors found for all reviews. In addition, the error density for each work product can be computed.

Once data are collected for many reviews conducted across many projects, average values for error density enable you to estimate the number of errors to be found in a new document before it is reviewed. For example, if the average error density for a requirements model is 0.68 errors per page, and a new requirements model is 40 pages long, a rough estimate suggests that your software team will find around 27 errors during the review of the document. If you find only 9 errors, you've either done an extremely good job in developing the requirements model or your review approach was not thorough enough.

It is difficult to measure the cost effectiveness of any technical review in real time. A software engineering organization can assess the effectiveness of reviews and their cost benefit only after reviews have been completed, review metrics have been collected, average data have been computed, and then the downstream quality of the software is measured (via testing).

Returning to the previous example, the average error density for requirements models was determined to be 0.68 per page. The effort required to correct a minor model error (immediately after the review) has been found to require 4 person-hours. The effort required for a major requirement error has been found to be 18 person-hours. Examining the review data collected, you find that minor errors occur about six times more frequently than major errors. Therefore, you can estimate that the average effort to find and correct a requirements error during review is about 6 person-hours.

Requirements-related errors uncovered during testing require an average of 45 person-hours to find and correct (no data are available on the relative severity of the error). Using the averages noted, we get:

$$\begin{aligned}\text{Effort saved per error} &= E_{\text{testing}} - E_{\text{reviews}} \\ &= 45 - 6 = 39 \text{ person-hours/error}\end{aligned}$$

Because 22 errors were found during the review of the requirements model, a savings of about 858 person-hours of testing effort would be achieved. And that's just for requirements-related errors. Errors associated with design and code would add to the overall benefit.

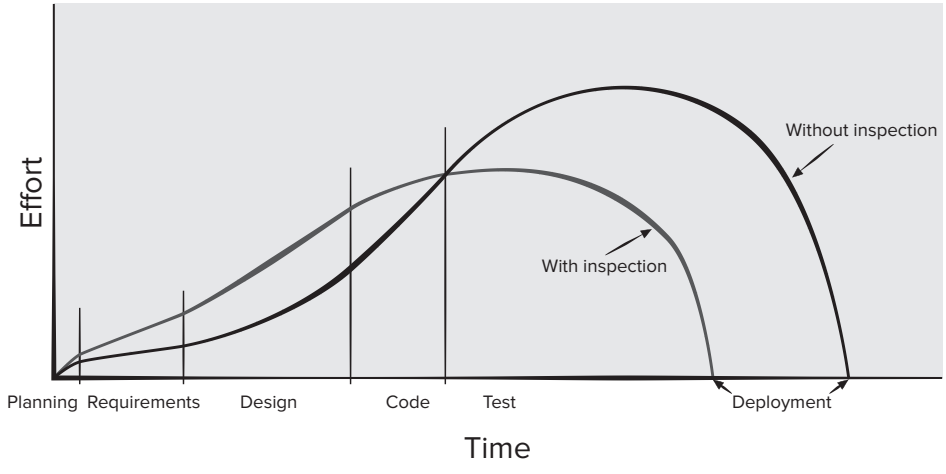
The bottom line—effort saved leads to shorter delivery cycles and improved time to market. The example presented in this section suggests this may be true. More importantly, industry data for software reviews has been collected for more than three decades and is summarized qualitatively using the graphs shown in Figure 16.1.

Referring to the figure, the effort expended when reviews are used does increase early in the development of a software increment, but this early investment for reviews pays dividends because testing and corrective effort is reduced. It is important to note

FIGURE 16.1

Effort expended with and without reviews

Source: Fagan, Michael E., "Advances in Software Inspections," IEEE Transactions on Software Engineering, vol. SE-12, no. 7, July 1986, 744–751.



that the deployment date for development with reviews is sooner than the deployment date without reviews. Reviews don't take time; they save it!

16.4 Criteria for Types of Reviews

Technical reviews can be classified as either formal or informal or somewhere in between these two extremes. The level of formality is chosen to match the type of product to be built, the project time line, and the people who are doing the work. Figure 16.2 depicts a reference model for technical reviews [Lai02] that identifies four characteristics that contribute to the formality with which a review is conducted.

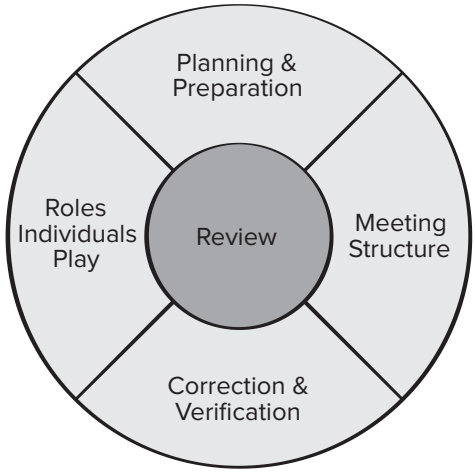
Each of the reference model characteristics helps to define the level of review formality. The formality of a review increases when (1) distinct roles are explicitly defined for the reviewers, (2) there is a sufficient amount of planning and preparation for the review, (3) a distinct structure for the review (including tasks and internal work products) is defined, and (4) follow-up by the reviewers occurs for any corrections that are made.

An element that is not presented in this model is the frequency of the reviews themselves. If you are using an agile prototyping model (Chapter 4) that contains relatively short sprints, your team may opt for less formal reviews because the reviews are happening fairly often. This usually means that defects are caught sooner and more often.

To understand the reference model, let's assume that you've decided to review the interface design for **SafeHomeAssured.com.** You can do this in a variety of different ways that range from relatively casual to extremely rigorous. If you decide that the casual approach is most appropriate, you ask a few colleagues (peers) to examine the interface prototype in an effort to uncover potential problems. All of you decide that there will be no advance preparation, but that you will evaluate the prototype in a reasonably structured way—looking at layout first, aesthetics next, navigation options after that, and so on. As the designer, you decide to take a few notes, but nothing formal.

FIGURE 16.2

Reference model for technical reviews



But what if the interface is pivotal to the success of the entire project? What if human lives depended on an interface that was ergonomically sound? You might decide that a more rigorous approach was necessary. A review team would be formed. Each person on the team would have a specific role to play—leading the team, recording findings, presenting the material, and so on. Each reviewer would be given access to the work product (in this case, the interface prototype) before the review and would spend time looking for errors, inconsistencies, and omissions. A set of specific tasks would be conducted based on an agenda that was developed before the review occurred. The results of the review would be formally recorded, and the team would decide on the status of the work product based on the outcome of the review. Members of the review team might also verify that the corrections made were done properly.

In this book we consider two broad categories of technical reviews: informal reviews and more formal technical reviews. Within each category, a number of different approaches can be chosen. These are presented in the sections that follow.

16.5 INFORMAL REVIEWS

Informal reviews include a simple desk check of a software engineering work product with a colleague, a casual meeting (involving more than two people) for the purpose of reviewing a work product, or the review-oriented aspects of pair programming (Chapter 3).

A simple *desk check* or a *casual meeting* conducted with a colleague is a review. However, because there is no advance planning or preparation, no agenda or meeting structure, and no follow-up on the errors that are uncovered, the effectiveness of such reviews is considerably lower than more formal approaches. But a simple desk check can and does uncover errors that might otherwise propagate further into the software process.

One way to improve the efficacy of a desk check review is to develop a set of simple review checklists[2] for each major work product produced by the software team. The questions posed within the checklist are generic, but they will serve to guide the reviewers as they check the work product. For example, let's reexamine a desk check of the interface prototype for **SafeHomeAssured.com.** Rather than simply playing with the prototype at the designer's workstation, the designer and a colleague examine the prototype using a checklist for interfaces:

- Is the layout designed using standard conventions? Left to right? Top to bottom?
- Does the presentation need to be scrolled?
- Are color and placement, typeface, and size used effectively?
- Are all navigation options or functions represented at the same level of abstraction?
- Are all navigation choices clearly labeled?

2 Literally hundreds of technical review checklists can be found via a web search. For example, a useful code review checklist can be downloaded from https://courses.cs.washington.edu/courses/cse403/12wi/sections/12wi_code_review_checklist.pdf.

and so on. Any errors or issues noted by the reviewers are recorded by the designer for resolution at a later time. Desk checks may be scheduled in an ad hoc manner, or they may be mandated as part of good software engineering practice. In general, the amount of material to be reviewed is relatively small and the overall time spent on a desk check spans little more than 1 or 2 hours.

In Chapter 3, we described *pair programming* in the following manner: XP recommends that two people work together at one computer workstation to create code for a story. This provides a mechanism for real-time problem solving (two heads are often better than one) and real-time quality assurance.

Pair programming (Section 3.5.1) can be characterized as a continuous desk check. Rather than scheduling a review at some point in time, pair programming encourages continuous review as a work product (design or code) is created. The benefit is immediate discovery of errors and better work product quality.

Some software engineers argue that the inherent redundancy built into pair programming is wasteful of resources. After all, why assign two people to a job that one person can accomplish? The answer to this question can be found in Section 16.3. If the quality of the work product produced as a consequence of pair programming is significantly better than the work of an individual, the quality-related savings can more than justify the "redundancy" implied by pair programming.

16.6 FORMAL TECHNICAL REVIEWS

A *formal technical review* (FTR) is a software quality control activity performed by software engineers (and others). The objectives of an FTR are: (1) to uncover errors in function, logic, or implementation for any representation of the software; (2) to verify that the software under review meets its requirements; (3) to ensure that the software has been represented according to predefined standards; (4) to achieve software that is developed in a uniform manner; and (5) to make projects more manageable. In addition, the FTR serves as a training ground, enabling junior engineers to observe different approaches to software analysis, design, and implementation. The FTR also serves to promote backup and continuity because a number of people become familiar with parts of the software that they may not have otherwise seen.

The FTR is actually a class of reviews that includes *walkthroughs* and *inspections*. Each FTR is conducted as a meeting and will be successful only if it is properly planned, controlled, and attended. In the sections that follow, guidelines similar to those for a walkthrough are presented as a representative formal technical review. If you have interest in software inspections, as well as additional information on walkthroughs, see [Rad02], [Wie02], or [Fre90].

16.6.1 The Review Meeting

Regardless of the FTR format that is chosen, every review meeting should abide by the following constraints:

- Between three and five people (typically) should be involved in the review.
- Advance preparation should occur but should require no more than 2 hours of work for each person.
- The duration of the review meeting should be less than 2 hours.

Given these constraints, it should be obvious that an FTR focuses on a specific (and small) part of the overall software. For example, rather than attempting to review an entire design, walkthroughs are conducted for each component or small group of components. By narrowing the focus, the FTR has a higher likelihood of uncovering errors.

The focus of the FTR is on a work product (e.g., a self-contained portion of a requirements model, a detailed component design, or source code for a component). The individual who has developed the work product—the *producer*—informs the project leader that the work product is complete and that a review is required. The project leader contacts a *review leader,* who evaluates the product for readiness, generates copies of product materials, and distributes them to two or three *reviewers* for advance preparation. Each reviewer is expected to spend between 1 and 2 hours reviewing the product, making notes, and otherwise becoming familiar with the work. Concurrently, the review leader also reviews the product and establishes an agenda for the review meeting, which is typically scheduled for the next day.

The review meeting is attended by the review leader, all reviewers, and the producer. One of the reviewers takes on the role of a *recorder,* that is, the individual who records (in writing) all important issues raised during the review. The FTR begins with an introduction of the agenda and a brief introduction by the producer. The producer then proceeds to "walk through" the work product, explaining the material, while reviewers raise issues based on their advance preparation. When valid problems or errors are discovered, the recorder notes each. Don't point out errors too harshly. One way to be gentle is to ask a question that enables the producer to discover the error.

At the end of the review, all attendees of the FTR must decide whether to: (1) accept the product without further modification, (2) reject the product due to severe errors (once corrected, another review must be performed), or (3) accept the product provisionally (minor errors have been encountered and must be corrected, but no additional review will be required). After the decision is made, all FTR attendees complete a sign-off, indicating their participation in the review and their concurrence with the review team's findings.

16.6.2 Review Reporting and Record Keeping

During the FTR, a reviewer (the recorder) actively records all issues that have been raised. These are summarized at the end of the review meeting, and a *review issues list* is produced. In addition, a *formal technical review summary report* is completed. A review summary report answers three questions:

1. What was reviewed?
2. Who reviewed it?
3. What were the findings and conclusions?

The review summary report is a single-page form (with possible attachments). It becomes part of the project historical record and may be distributed to the project leader and other interested parties.

The review issues list serves two purposes: (1) to identify problem areas within the product and (2) to serve as an action item checklist that guides the producer as corrections are made. An issues list is normally attached to the summary report.